

VAST Admin Commands

Complete Reference — Modules 1 to 9 + Capstone

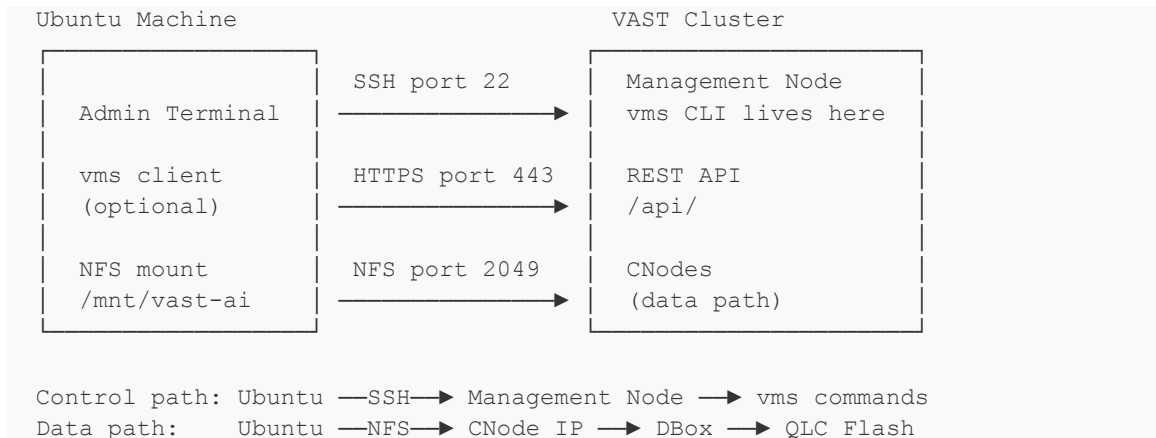
NFS • SMB • S3 • Performance • Troubleshooting • Log Analysis

Qualcomm Delivery | August 2026

IMPORTANT: Every vms command in this document is from VAST official documentation. Each command group has a REST API alternative (orange) that always works via port 443 — use this if vms CLI is unavailable.

Module	Topic	Key VAST Commands
1	Storage Architecture	vms cluster show, vms cnodes list, vms dboxes list, vms volumes, vms views
2	Monitoring Fundamentals	vms alerts list, vms events list, vms metrics get
3	Storage Performance	vms metrics get, vms qospolicies, fio from Ubuntu client
4	Network Performance	vms cnodes show, vms nics list, iperf3 from Ubuntu client
5	Protocol Performance — NFS	vms views create --protocols NFS, nfsstat, mountstats
5	Protocol Performance — SMB	vms smbshares list/create, vms users, smbclient
6	Performance Optimization	vms qospolicies, vms protectionpolicies, vms volumes modify
7	Storage Service Troubleshooting	vms alerts, vms events, vms drives, vms supportbundles
8	Performance Troubleshooting	vms metrics get, vms quotas, end-to-end diagnosis
9	Log Analysis & Diagnostics	vms events, vms snapshots, vms supportbundles, DR commands
Capstone 1	AI Training Degradation	Full VAST diagnosis + fix commands
Capstone 2	AI Dataset Access Failure	Full VAST diagnosis + fix commands

HOW TO CONNECT UBUNTU TO VAST — FIRST STEP



Method 1: SSH to Management Node (Standard)

```
# SSH to VAST management node
# Get <mgmt-ip> from your VAST team
ssh vastadmin@<mgmt-ip>

# Once logged in - verify vms works
vms --help
vms cluster show
```

✓ EXPECTED OUTPUT: VAST cluster name, version, health state, capacity summary

Method 2: REST API from Ubuntu (No SSH Needed)

```
# Set once - use throughout all modules
export VAST_IP='<cluster-mgmt-ip>'
export VAST_USER='admin'
export VAST_PASS='<password>'

# Test connection
curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/clusters/ | python3 -m json.tool
```

✓ EXPECTED OUTPUT: JSON: cluster name, version, state: ONLINE, capacity fields

Mount VAST NFS from Ubuntu — Data Path

```
# Get CNode IP from: vms cnodes list
# Get export path from: vms views show --name <view-name>

sudo mkdir -p /mnt/vast-ai

# Mount with AI-optimised options
sudo mount -t nfs \
  -o vers=3,nconnect=16,rsz=1048576,wsz=1048576,noatime,hard,proto=tcp \
  <cnode-ip>:/ai_training /mnt/vast-ai
```

```
# Verify
findmnt /mnt/vast-ai
df -h /mnt/vast-ai
```

✓ EXPECTED OUTPUT: /mnt/vast-ai <cnode-ip>:/ai_training rw,nconnect=16,rspace=1048576

MODULE 1 — Storage Architecture & VAST Platform Overview

VAST DASE Architecture:

CNode 1	CNode 2	CNode 3	CNode 4	(Stateless)
NFS/SMB/S3 endpoints — any CNode serves any client				
↑ NVMe-oF over RDMA				
DBox 1	DBox 2	DBox 3	DBox 4	(Flash Storage)
QLC Flash — shared equally by all CNodes				

Volume = logical data container (/ai_training)

View = protocol access door (NFS export / SMB share)

Policy = rules per view (QoS / snapshot / protection)

Cluster Commands

 **PURPOSE:** Get full cluster overview — first command every VAST admin runs.

```
# Full cluster overview
vms cluster show

# List clusters (multi-cluster environments)
vms cluster list
```

✓ **EXPECTED OUTPUT:** name | state: ONLINE | sw_version | physical_capacity | logical_capacity | cnodes_count | dboxes_count

 **REST API:** `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/clusters/` | `python3 -m json.tool`

CNode Commands

 **PURPOSE:** Inspect compute nodes — stateless servers that serve NFS/SMB/S3 to clients.

```
# List all CNodes
vms cnodes list

# Show specific CNode — note the IP address for client mounts
vms cnodes show --id 1
```

✓ **EXPECTED OUTPUT:** id | hostname | ip | state: ONLINE | for each CNode

 **REST API:** `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/cnodes/` | `python3 -m json.tool`

DBox Commands

 **PURPOSE:** Inspect flash storage enclosures. All must be ONLINE for full performance.

```
# List all DBoxes
vms dboxes list

# Show specific DBox
vms dboxes show --id 1

# List all drives inside DBoxes
vms drives list
```

```
# Show specific drive
vms drives show --id 1
```

✓ EXPECTED OUTPUT: id | hostname | state: ONLINE | capacity | for each DBox and drive

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/dboxes/ | python3 -m json.tool`

Volume Commands

🎯 PURPOSE: Volumes are logical data containers. Create one per team or project.

```
# List all volumes
vms volumes list

# Show volume details
vms volumes show --name ai_training

# Create volume
vms volumes create --name ai_training --path /ai_training

# Modify volume
vms volumes modify --name ai_training --snapshot-policy daily_snap

# Delete volume
vms volumes delete --name ai_training
```

✓ EXPECTED OUTPUT: id | name | path | physical_used | logical_used | quota

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/folders/ | python3 -m json.tool`

View Commands — NFS / SMB / S3 Exports

🎯 PURPOSE: Views are the doors clients use. One volume can have NFS, SMB, and S3 views simultaneously.

```
# List all views
vms views list

# Show view details
vms views show --name training_view

# Create NFS view
vms views create --name nfs_view --path /ai_training --protocols NFS

# Create SMB view
vms views create --name smb_view --path /ai_training --protocols SMB

# Create multi-protocol view
vms views create --name multi_view --path /ai_training --protocols NFS,SMB,S3

# Modify view — add QoS policy
vms views modify --name nfs_view --qos-policy ai_qos

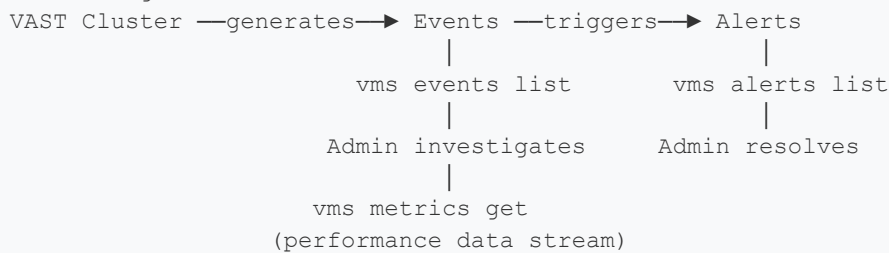
# Delete view
vms views delete --name nfs_view
```

✓ EXPECTED OUTPUT: id | name | path | protocols | nfs_path | smb_share_name | for each view

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/views/ | python3 -m json.tool`

MODULE 2 — Monitoring Fundamentals

Monitoring Flow:



Alert Commands

 **PURPOSE:** Alerts are the primary signal that something needs attention. Check every morning.

```
# List ALL active (unresolved) alerts
vms alerts list

# Filter by severity
vms alerts list --severity CRITICAL
vms alerts list --severity WARNING
vms alerts list --severity INFO

# List resolved alerts
vms alerts list --resolved true

# Show specific alert details
vms alerts show --id 1
```

 **EXPECTED OUTPUT:** id | severity | title | description | affected_object | created_at | resolved_at

 **REST API:** `curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/alerts/?resolved=false' | python3 -m json.tool`

Event Commands

 **PURPOSE:** Events are the detailed log of everything that happened in the cluster. Use for root cause analysis.

```
# List all recent events
vms events list

# Filter by severity
vms events list --severity INFO
vms events list --severity WARNING
vms events list --severity ERROR
vms events list --severity CRITICAL

# Filter by time range
vms events list --from 2026-08-09T00:00:00 --to 2026-08-09T23:59:59

# Filter by object type
vms events list --object-type CNODE
vms events list --object-type DBOX
vms events list --object-type DRIVE
```

✓ EXPECTED OUTPUT: id | severity | object_type | description | timestamp | for each event

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/events/?severity=ERROR' | python3 -m json.tool`

Metrics Commands

🎯 PURPOSE: Get real-time performance counters — IOPS, throughput, latency — for the whole cluster.

```
# Get current cluster metrics
vms metrics get

# Get metrics for last hour
vms metrics get --from 2026-08-09T09:00:00 --to 2026-08-09T10:00:00

# Export metrics as CSV
vms metrics get --format csv > /tmp/vast_metrics.csv
```

✓ EXPECTED OUTPUT: timestamp | rd_iops | wr_iops | rd_bw_mbps | wr_bw_mbps | rd_lat_ms | wr_lat_ms

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/monitors/ad_hoc_query/' -X POST -H 'Content-Type: application/json' -d '{"object_type":"cluster","time_frame":"last_hour","prop_list":["rd_iops","wr_iops","rd_bw","wr_bw","rd_lat","wr_lat"]}' | python3 -m json.tool`

Health Validation — Morning Checklist

🎯 PURPOSE: Run every morning before training or production workloads start.

```
echo '===== '
echo ' VAST Morning Health Check'
echo ' $(date)'
echo '===== '

echo '[1] Cluster State'
vms cluster show

echo '[2] Active Alerts'
vms alerts list --severity CRITICAL
vms alerts list --severity WARNING

echo '[3] CNode Health'
vms cnodes list

echo '[4] DBox Health'
vms dboxes list

echo '[5] Drive Health'
vms drives list

echo '[6] Error Events (last 24h)'
vms events list --severity ERROR

echo '[7] Performance Metrics'
vms metrics get
```



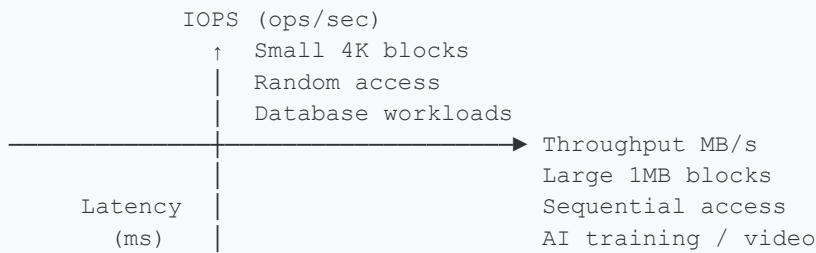
```
echo '===== '  
echo ' Check Complete '  
echo '===== '
```

✅ **EXPECTED OUTPUT: All components ONLINE | No CRITICAL alerts | No ERROR events | Metrics within baseline**

💡 **KEY LESSON: If ANY CNode or DBox shows state other than ONLINE — raise an alert immediately. VAST is designed for zero downtime but degraded components reduce redundancy.**

MODULE 3 — Storage Performance Fundamentals

Performance Triangle:



Queue Depth effect:

QD=1: —[req]—————[resp]—[req]— (serial, low IOPS)
QD=32: —[r][r][r][r][r]—[res*32]— (parallel, high IOPS)

Performance Metrics on VAST

🎯 **PURPOSE:** Get IOPS, throughput, and latency from VAST cluster to establish baseline.

```
# Get current cluster performance
vms metrics get

# Get volume-level performance details
vms volumes list
vms volumes show --name ai_training

# Get quota usage (capacity performance)
vms quotas list
```

✅ **EXPECTED OUTPUT:** rd_iops | wr_iops | rd_bw_mbps | wr_bw_mbps | rd_lat_ms | wr_lat_ms

Performance Baseline — Ubuntu to VAST

🎯 **PURPOSE:** Measure actual delivered performance from Ubuntu client to VAST. Establish baseline before tuning.

```
# Sequential write throughput (AI training write pattern)
echo '=== Sequential Write Baseline ==='
fio --name=seq_write --rw=write --bs=1M --size=1G \
    --directory=/mnt/vast-ai --numjobs=1 --group_reporting \
    2>&1 | grep -E 'bw=|iops='

# Sequential read throughput (AI training read pattern)
echo '=== Sequential Read Baseline ==='
fio --name=seq_read --rw=read --bs=1M --size=1G \
    --directory=/mnt/vast-ai --numjobs=1 --group_reporting \
    2>&1 | grep -E 'bw=|iops='

# Random read IOPS (database pattern)
echo '=== Random Read IOPS ==='
fio --name=rand_read --rw=randread --bs=4k --size=1G \
    --directory=/mnt/vast-ai --iodepth=32 --numjobs=4 --group_reporting \
    2>&1 | grep -E 'bw=|iops='

# Queue depth comparison
```

```

echo '=== Queue Depth 1 ==='
fio --name=qd1 --rw=randread --bs=4k --size=512M \
    --directory=/mnt/vast-ai --iodepth=1 --numjobs=1 --group_reporting \
    2>&1 | grep -E 'iops=|lat='

echo '=== Queue Depth 32 ==='
fio --name=qd32 --rw=randread --bs=4k --size=512M \
    --directory=/mnt/vast-ai --iodepth=32 --numjobs=4 --group_reporting \
    2>&1 | grep -E 'iops=|lat='

```

✓ EXPECTED OUTPUT: seq_write: XXXX MB/s | seq_read: XXXX MB/s | rand_read: XXXX IOPS | QD32 shows higher IOPS than QD1

QoS Policy Commands

🎯 PURPOSE: Control per-workload performance limits on VAST. Protect latency-sensitive workloads from throughput-hungry ones.

```

# List QoS policies
vms qospolicies list

# Create AI training QoS policy (high bandwidth)
vms qospolicies create \
    --name ai_training_qos \
    --max-throughput-mbps 10000 \
    --max-iops 500000

# Create interactive QoS policy (low latency priority)
vms qospolicies create \
    --name interactive_qos \
    --max-throughput-mbps 2000 \
    --max-iops 100000

# Attach QoS to view
vms views modify --name ai_view --qos-policy ai_training_qos

# Show QoS policy
vms qospolicies show --name ai_training_qos

# Delete QoS policy
vms qospolicies delete --name ai_training_qos

```

✓ EXPECTED OUTPUT: id | name | max_throughput_mbps | max_iops | attached_views

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/qospolicies/` | `python3 -m json.tool`

MODULE 4 — Network Performance Analysis

Network Path: Ubuntu Client → Switch → VAST CNode → DBox

Packet Flow Analysis:

Client sends READ request → [1ms] → CNode receives
CNode fetches data from DBox → [<0.1ms NVMe-oF] → CNode
CNode sends data to Client → [1ms] → Client receives
Total latency = network RTT + CNode processing + NVMe-oF

Congestion points:

1. Client NIC saturated → check: `ethtool eth0`
2. Switch oversubscribed → check: `ping packet loss`
3. CNode NIC saturated → check: `vms cnodes show --id N`
4. `nconnect` not set → check: `ss -tn dst :2049 | wc -l`

VAST Network Commands

 **PURPOSE:** Inspect CNode network interfaces and confirm connectivity.

```
# List all CNodes with IPs - get IPs for client mounts
vms cnodes list

# Show CNode network details
vms cnodes show --id 1

# List all network interfaces on cluster
vms nics list

# Show specific NIC
vms nics show --id 1
```

 **EXPECTED OUTPUT:** CNode IPs | NIC names | link speed | link state | for all interfaces

 **REST API:** `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/nics/ | python3 -m json.tool`

Ubuntu Network Validation to VAST

 **PURPOSE:** Test all network layers from Ubuntu to VAST CNodes before blaming storage.

```
# Step 1: Basic reachability
ping -c 20 <cnode-ip> | tail -3

# Step 2: NFS port open on CNode
nc -zv <cnode-ip> 2049 && echo 'NFS port OPEN'
nc -zv <cnode-ip> 111 && echo 'RPC port OPEN'

# Step 3: Bandwidth test to CNode
# Run iperf3 server on CNode or another Ubuntu machine
iperf3 -c <cnode-ip> -t 30 -P 8 2>&1 | tail -4

# Step 4: Check NFS connection count
ss -tn dst :2049 | wc -l | xargs echo 'NFS TCP connections:'

# Step 5: Check for packet loss
```

```
ping -c 1000 -i 0.01 <cnode-ip> 2>&1 | tail -3
```

```
# Step 6: Network interface stats on Ubuntu
ethtool eth0 | grep -E 'Speed|Duplex|Link'
ethtool -S eth0 | grep -E 'error|drop|miss'
```

✅ EXPECTED OUTPUT: ping < 1ms | port OPEN | iperf3 > 10 Gbits | NFS connections = 16 | 0% packet loss

Congestion Detection

🎯 PURPOSE: Detect network congestion before it impacts storage performance.

```
# Check NFS retransmits - most critical metric
# retrans > 0 = network problem
nfsstat -c 2>/dev/null | head -6

# Check Ubuntu NIC errors
netstat -s | grep -iE 'retransmit|error|fail'

# Simulate network delay to understand impact
sudo tc qdisc del dev lo root 2>/dev/null; echo 'tc cleared'
sudo tc qdisc add dev lo root netem delay 20ms
fio --name=delay_test --rw=read --bs=1M --size=256M \
    --directory=/mnt/vast-ai --numjobs=1 --group_reporting \
    2>&1 | grep -E 'bw=|lat='
sudo tc qdisc del dev lo root && echo 'delay removed'
```

✅ EXPECTED OUTPUT: retrans=0 (healthy) | any retrans > 0 = investigate network | delay test shows latency impact

MODULE 5 — Protocol Performance Analysis

PART A: NFS Protocol

NFS Architecture on VAST:

Ubuntu Client —NFS v3/v4→ VAST CNode (NFS service)

|
Authentication
UID/GID mapping
Export policy check
|
Data served from DBox

NFSv3: Stateless — server restart is transparent to client

NFSv4: Stateful — sessions, delegations, better locking

Key option: nconnect=16 — opens 16 TCP connections per client

NFS View Configuration on VAST

 **PURPOSE:** Create and configure NFS exports on VAST.

```
# Create NFSv3 export
vms views create \
  --name nfs3_training \
  --path /ai_training \
  --protocols NFSv3

# Create NFSv4 export
vms views create \
  --name nfs4_training \
  --path /ai_training \
  --protocols NFSv4

# Create export supporting both v3 and v4
vms views create \
  --name nfs_all \
  --path /ai_training \
  --protocols NFS

# List all NFS views
vms views list

# Show NFS view details — get the export path for clients
vms views show --name nfs3_training
```

 **EXPECTED OUTPUT:** id | name | path | protocols: NFSv3 | nfs_path: /ai_training

 **REST API:** curl -sk -u \$VAST_USER:\$VAST_PASS 'https://\$VAST_IP/api/views/?protocols=NFS' | python3 -m json.tool

NFS Mount Options — Ubuntu Client

 **PURPOSE:** Mount VAST NFS with correct options. Wrong options = half the performance.

Option	Value	Purpose	Impact if Missing
nconnect	16	Open 16 TCP connections	16x bandwidth reduction
rsize	1048576	1MB read block	4x more round trips
wsiz	1048576	1MB write block	4x more round trips
noatime	(flag)	Skip access time writes	Extra metadata per read
hard	(flag)	Retry forever on failure	Data corruption risk
proto=tcp	tcp	Use TCP not UDP	Less reliable on 10GbE+
vers=3	3	NFSv3 stateless	Server restart = hang

```
# Optimised NFS mount for AI training workloads
sudo mount -t nfs \
  -o vers=3,nconnect=16,rsize=1048576,wsiz=1048576,noatime,hard,proto=tcp \
  <cnode-ip>:/ai_training /mnt/vast-ai

# Verify mount options are applied
findmnt /mnt/vast-ai -o TARGET,OPTIONS

# Check how many TCP connections opened
ss -tn dst :2049 | wc -l | xargs echo 'TCP connections to VAST:'

# Check NFS server exports
showmount -e <cnode-ip>

# Check NFS client statistics
nfsstat -c 2>/dev/null | head -8

# Per-mount latency breakdown
mountstats /mnt/vast-ai 2>/dev/null | head -30

# Live NFS IOPS and latency
nfsiostat 2
```

✓ EXPECTED OUTPUT: Options: nconnect=16,rsize=1048576 | TCP connections: 16 | retrans=0 | latency < 1ms

NFS Performance Analysis

🎯 PURPOSE: Measure and compare NFS performance across different options.

```
# NFSv3 throughput
echo '=== NFSv3 Throughput ==='
fio --name=nfs3 --rw=read --bs=1M --size=1G \
  --directory=/mnt/vast-nfs3 --numjobs=4 --group_reporting \
  2>&1 | grep bw=

# NFSv4 throughput
echo '=== NFSv4 Throughput ==='
fio --name=nfs4 --rw=read --bs=1M --size=1G \
  --directory=/mnt/vast-nfs4 --numjobs=4 --group_reporting \
  2>&1 | grep bw=

# nconnect=1 vs nconnect=16
echo '=== Single connection ==='
fio --name=single --rw=read --bs=1M --size=1G \
  --directory=/mnt/single --numjobs=1 --group_reporting \
  2>&1 | grep bw=
```

```
echo '=== 16 connections ==='
fio --name=multi --rw=read --bs=1M --size=1G \
    --directory=/mnt/vast-ai --numjobs=1 --group_reporting \
    2>&1 | grep bw=
```

✓ EXPECTED OUTPUT: NFSv3 vs NFSv4 comparison | single: low MB/s | 16 connections: much higher MB/s

MODULE 5 — Protocol Performance Analysis

PART B: SMB Protocol

SMB Architecture on VAST:

```
Windows/Linux Client —SMB3→ VAST CNode (SMB service)
                                |
                                Active Directory
                                Kerberos auth
                                ACL permission check
                                |
                                Data served from DBox
```

SMB3 features: multichannel, encryption, persistent handles

Auth options: Kerberos (AD joined), NTLMv2, anonymous

VAST SMB: same data accessible via NFS + SMB simultaneously

SMB Share Commands on VAST

 **PURPOSE:** Create and manage SMB shares on VAST for Windows clients.

```
# List all SMB shares
vms smbshares list

# Show specific SMB share details
vms smbshares show --name winshare

# Create SMB share from existing view
# First create the view, then SMB share points to same path
vms views create \
  --name smb_training \
  --path /ai_training \
  --protocols SMB

# List views with SMB protocol
vms views list

# Show SMB view — get the share name for Windows clients
vms views show --name smb_training
```

 **EXPECTED OUTPUT:** id | name | path | protocols: SMB | smb_share_name for Windows UNC path

 **REST API:** `curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/views/?protocols=SMB'`
| `python3 -m json.tool`

User and Authentication Commands

 **PURPOSE:** Manage VAST users and groups for SMB authentication. Also applies to NFS UID/GID mapping.

```
# List all users
vms users list

# Show specific user
vms users show --name admin
```

```
# Create local user (for SMB auth without AD)
vms users create --name team_a_user --password SecurePass123

# List user groups
vms usergroups list

# Create user group
vms usergroups create --name ai_team

# Add user to group
vms usergroups modify --name ai_team --add-users team_a_user

# Show group details
vms usergroups show --name ai_team
```

✓ EXPECTED OUTPUT: id | name | email | groups | for each user

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/users/ | python3 -m json.tool`

SMB Access Validation from Ubuntu

🎯 PURPOSE: Test SMB connectivity from Ubuntu using smbclient — works without Windows machine.

```
# List shares on VAST CNode via SMB
smbclient -L //<cnode-ip> -U team_a_user

# Connect to specific SMB share
smbclient //<cnode-ip>/smb_training -U team_a_user

# Mount SMB share on Ubuntu
sudo mkdir -p /mnt/vast-smb
sudo mount -t cifs //<cnode-ip>/smb_training /mnt/vast-smb \
-o username=team_a_user,password=SecurePass123,vers=3.0

# Verify SMB mount
findmnt /mnt/vast-smb
df -h /mnt/vast-smb

# Test write via SMB
touch /mnt/vast-smb/.test && echo 'SMB write OK' && rm /mnt/vast-smb/.test

# Measure SMB throughput
fio --name=smb_test --rw=write --bs=1M --size=512M \
--directory=/mnt/vast-smb --numjobs=1 --group_reporting \
2>&1 | grep bw=
```

✓ EXPECTED OUTPUT: Shares listed | SMB mounted | Write OK | throughput MB/s

Access Audit — Who Accessed What via Which Protocol

🎯 PURPOSE: Track all access activity across NFS and SMB. Essential for multi-team environments.

```
# List audit events
vms audits list
```

```
# Filter by protocol
vms audits list --protocol NFS
vms audits list --protocol SMB

# Filter by path
vms audits list --path /ai_training

# Filter by time
vms audits list --from 2026-08-09T00:00:00
```

✓ EXPECTED OUTPUT: timestamp | user | protocol | operation | path | result: SUCCESS/DENY

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/audits/?protocol=SMB' | python3 -m json.tool`

MODULE 6 — Performance Optimization

Optimization Process:

```
Measure baseline → Identify bottleneck → Apply fix → Verify improvement
|               |               |               |
fio baseline    nfsstat retrans  remount/QoS    fio after
vms metrics get ss connections  vms modify     vms metrics
```

Optimization Layers (fix from bottom up):

```
Layer 1: Mount options  (nconnect, rsize, noatime)
Layer 2: QoS policy     (per-view bandwidth limit)
Layer 3: Data reduction (off for already-compressed data)
Layer 4: Quota          (ensure not hitting limit)
```

Baseline Development

 **PURPOSE:** Establish performance baseline before any tuning. You cannot improve what you have not measured.

```
# Capture baseline — save to file for comparison
echo '=== VAST Performance Baseline ===' > /tmp/vast_baseline.txt
echo 'Date: '$(date) >> /tmp/vast_baseline.txt

# Baseline 1: Sequential throughput
fio --name=baseline_seq --rw=read --bs=1M --size=1G \
    --directory=/mnt/vast-ai --numjobs=4 --group_reporting \
    2>&1 | grep bw= >> /tmp/vast_baseline.txt

# Baseline 2: Random IOPS
fio --name=baseline_rand --rw=randread --bs=4k --size=512M \
    --directory=/mnt/vast-ai --iodepth=32 --numjobs=4 --group_reporting \
    2>&1 | grep iops= >> /tmp/vast_baseline.txt

# Baseline 3: NFS connections
ss -tn dst :2049 | wc -l >> /tmp/vast_baseline.txt

# Baseline 4: Retransmits
nfsstat -c 2>/dev/null | head -4 >> /tmp/vast_baseline.txt

cat /tmp/vast_baseline.txt
```

 **EXPECTED OUTPUT:** Baseline file with: seq throughput MB/s | rand IOPS | connections=16 | retrans=0

Protection Policy — Data Reduction Tuning

 **PURPOSE:** Disable compression for AI model files (already compressed). Saves CPU, improves throughput.

```
# List protection policies
vms protectionpolicies list

# Create policy — NO reduction for AI .pt files
# AI model files (.pt .onnx .bin) are already compressed
# Trying to compress them wastes CPU with no benefit
```

```
vms protectionpolicies create \
  --name ai_no_reduction \
  --compression false \
  --dedup false

# Create policy - MAX reduction for logs and text
vms protectionpolicies create \
  --name logs_max_reduction \
  --compression true \
  --dedup true

# Attach to volume
vms volumes modify --name ai_training --protection-policy ai_no_reduction

# Show current reduction ratio
vms volumes show --name ai_training
```

✓ EXPECTED OUTPUT: Policy created | Volume updated | reduction_ratio shows improvement

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/protectionpolicies/ | python3 -m json.tool`

Resource Optimization — Quota Management

🎯 PURPOSE: Prevent one team from consuming all storage. Quotas protect shared environments.

```
# List all quotas
vms quotas list

# Create quota for AI team
vms quotas create \
  --name team_a_quota \
  --path /ai_training/team_a \
  --hard-limit 10TB \
  --soft-limit 9TB

# Show quota usage
vms quotas show --name team_a_quota

# Increase quota when needed
vms quotas modify --name team_a_quota --hard-limit 20TB

# Delete quota
vms quotas delete --name team_a_quota
```

✓ EXPECTED OUTPUT: id | name | path | hard_limit | soft_limit | used | used_percentage

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/quotas/ | python3 -m json.tool`

MODULE 7 — Storage Service Troubleshooting

Incident Response Workflow:

Report received

|

Step 1: Scope — one user or all users?

Step 2: Network — can client reach CNode?

Step 3: Mount — is NFS/SMB mount present?

Step 4: Write — can we read/write?

Step 5: VAST health — alerts? CNode/DBox ONLINE?

Step 6: Auth — permission denied? UID/GID mismatch?

Step 7: Namespace — correct export path?

Step 8: Logs — events, kernel messages

Step 9: Escalate with support bundle

Service Availability Commands

 **PURPOSE:** Quickly determine if the VAST cluster itself is the problem.

```
# Step 1: Cluster health
vms cluster show

# Step 2: Critical alerts
vms alerts list --severity CRITICAL

# Step 3: CNode status
vms cnodes list

# Step 4: DBox status
vms dboxes list

# Step 5: Drive status
vms drives list

# Step 6: Recent error events
vms events list --severity ERROR
```

 **EXPECTED OUTPUT:** Any non-ONLINE state or CRITICAL alert = cluster issue | All ONLINE + no alerts = client-side issue

Export Access Failure Commands

 **PURPOSE:** Diagnose why a client cannot access an NFS or SMB export.

```
# Verify the view (export) exists
vms views list
vms views show --name <view-name>

# Verify volume exists and has data
vms volumes show --name <volume-name>

# Check quota — is volume at 100%?
vms quotas list
vms quotas show --name <quota-name>
```

```
# From Ubuntu: test connectivity
showmount -e <cnode-ip>
nc -zv <cnode-ip> 2049 && echo 'NFS OPEN'

# From Ubuntu: test mount and write
findmnt -t nfs,nfs4 -o TARGET,SOURCE
touch /mnt/vast-ai/.test 2>&1 && echo 'Write OK' && rm /mnt/vast-ai/.test
```

✓ EXPECTED OUTPUT: View exists | Volume exists | Quota not exceeded | NFS port open | Write OK

Authentication and Permission Failures

🎯 PURPOSE: Diagnose auth failures — the most common cause of 'permission denied' on VAST.

```
# Check user exists on VAST
vms users list
vms users show --name <username>

# Check user group membership
vms usergroups list
vms usergroups show --name <groupname>

# Check audit log for permission denials
vms audits list --path /ai_training

# From Ubuntu: check your UID/GID
id

# From Ubuntu: check directory permissions on mount
ls -ld /mnt/vast-ai
ls -la /mnt/vast-ai/

# From Ubuntu: test as specific user
sudo -u team_a_user ls /mnt/vast-ai/

# From Ubuntu: check SMB auth
smbclient //<cnode-ip>/smb_training -U team_a_user -c 'ls'
```

✓ EXPECTED OUTPUT: User exists | Group membership confirmed | Audit shows deny reason | UID matches

Namespace Access Issues

🎯 PURPOSE: Diagnose when clients see wrong files or cannot traverse the directory tree.

```
# Verify export path on VAST
vms views show --name <view-name>

# Verify volume path
vms volumes show --name <volume-name>

# From Ubuntu: verify mount point
findmnt /mnt/vast-ai -o TARGET,SOURCE,OPTIONS

# From Ubuntu: list directory contents
ls -la /mnt/vast-ai/
```

```
# From Ubuntu: check showmount
showmount -e <cnode-ip>

# Stale mount recovery
sudo umount -f -l /mnt/vast-ai
sudo mount -t nfs \
    -o vers=3,nconnect=16,rsiz=1048576,wsiz=1048576,noatime,hard \
    <cnode-ip>:/ai_training /mnt/vast-ai
ls /mnt/vast-ai/ && echo 'Namespace accessible'
```

✓ EXPECTED OUTPUT: View path matches | Volume path matches | Namespace accessible after remount

Support Bundle for Escalation

🎯 PURPOSE: Create support bundle when issue needs VAST support team. Packages all cluster logs.

```
# Create support bundle
vms supportbundles create --reason 'Service outage on training cluster
2026-08-09'

# List bundles
vms supportbundles list

# Download bundle
vms supportbundles download --id 1 --output /tmp/vast_support_bundle.tar.gz

# Verify bundle downloaded
ls -lh /tmp/vast_support_bundle.tar.gz
```

✓ EXPECTED OUTPUT: Bundle created | id: N | Download complete | Share with VAST support team

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS -X POST https://$VAST_IP/api/supportbundles/ -H 'Content-Type: application/json' -d '{"reason":"service outage"}' | python3 -m json.tool`

MODULE 8 — Performance Troubleshooting

End-to-End Performance Investigation:

Application slow

Is GPU/CPU waiting for storage?

YES → Storage is bottleneck

Network issue?

nfsstat retrans > 0?

ping high latency?

packet loss?

Fix network first

Verify: nfsstat retrans=0

VAST cluster issue?

vms alerts list --severity CRITICAL

vms cnodes list (all ONLINE?)

vms dboxes list (all ONLINE?)

Mount options wrong?

nconnect = 1?

rsize = 256KB?

quota exceeded?

Remount + tune

Verify: fio shows improvement

Performance Metrics — VAST Side

🎯 **PURPOSE:** Pull VAST cluster metrics to see if the cluster is delivering expected performance.

```
# Current cluster performance
vms metrics get

# Check volume performance and usage
vms volumes show --name ai_training

# Check quota - capacity issue?
vms quotas show --name team_a_quota

# Check QoS - is policy throttling the workload?
vms qospolicies list
vms qospolicies show --name ai_training_qos

# Check alerts - any performance-related alerts?
vms alerts list --severity WARNING
vms alerts list --severity CRITICAL
```

✅ **EXPECTED OUTPUT:** Metrics: IOPS/BW/Latency | Quota usage % | QoS limits | Active alerts

Ubuntu End-to-End Performance Diagnosis

🎯 **PURPOSE:** Systematic step-by-step diagnosis from Ubuntu client to VAST cluster.

```
echo '===== '
echo ' End-to-End Performance Diagnosis '
echo '===== '
```

```

echo ''
echo '[Step 1] NFS Connection Count'
# count=1 = nconnect missing = bandwidth capped
ss -tn dst :2049 | wc -l | xargs echo 'TCP connections:'

echo ''
echo '[Step 2] NFS Retransmit Check'
# retrans > 0 = network problem - fix network first
nfsstat -c 2>/dev/null | head -6 || echo 'NFS not mounted'

echo ''
echo '[Step 3] Network Latency'
ping -c 10 <cnode-ip> 2>/dev/null | tail -2 || echo 'ping not available'

echo ''
echo '[Step 4] Measure Current Throughput'
fio --name=diag --rw=read --bs=1M --size=512M \
    --directory=/mnt/vast-ai --numjobs=4 --group_reporting \
    2>&l | grep bw=

echo ''
echo '[Step 5] Mount Options Check'
findmnt /mnt/vast-ai -o TARGET,OPTIONS 2>/dev/null || echo 'Not mounted'

echo ''
echo '[Step 6] Capacity Check'
df -h /mnt/vast-ai 2>/dev/null | tail -1
USED=$(df /mnt/vast-ai 2>/dev/null | awk 'NR==2{print $5}' | tr -d '%')
[ -n '$USED' ] && [ '$USED' -gt 90 ] && echo 'ALERT: Capacity ${USED}%' || echo
'Capacity OK: ${USED}%'

echo '===== '
echo ' Diagnosis Complete'
echo '===== '

```

✓ EXPECTED OUTPUT: Step 1: 16 | Step 2: retrans=0 | Step 3: < 1ms | Step 4: high MB/s | Step 5: options correct | Step 6: capacity OK

Workload Distribution Analysis

🎯 PURPOSE: Check if workload is hitting one CNode or balanced across all CNodes.

```

# Check all CNodes are serving traffic
vms cnodes list

# Check metrics (aggregated across all CNodes)
vms metrics get

# From Ubuntu: check which CNode IP is being used
ss -tn dst :2049 -o

# Network latency to each CNode
for cnode_ip in <cnode1-ip> <cnode2-ip> <cnode3-ip>; do
    ping -c 3 $cnode_ip 2>/dev/null | tail -1 | xargs echo "$cnode_ip:"
done

```

✓ EXPECTED OUTPUT: All CNodes ONLINE | Balanced metrics | Equal latency to all CNodes

MODULE 9 — Log Analysis and Diagnostics

Log Sources for Incident Investigation:

VAST Cluster Side:

```
vms events list
vms alerts list
vms audits list
vms supportbundles
```

Ubuntu Client Side:

```
dmesg -T | grep nfs
/var/log/syslog
nfsstat -c (retransmits)
mountstats /mnt/vast-ai
```

Correlate
Build timeline
Find root cause

Event and Alert Analysis

 **PURPOSE:** Build an incident timeline from VAST cluster events.

```
# All events in incident window
vms events list --from 2026-08-09T10:00:00 --to 2026-08-09T12:00:00

# All severity levels during incident
vms events list --severity ERROR --from 2026-08-09T10:00:00
vms events list --severity CRITICAL --from 2026-08-09T10:00:00

# All alerts that fired during incident
vms alerts list --from 2026-08-09T10:00:00

# Audit trail - what operations happened
vms audits list --from 2026-08-09T10:00:00
vms audits list --path /ai_training --from 2026-08-09T10:00:00
```

 **EXPECTED OUTPUT:** Timeline of events showing: what failed, when, which component, severity

 **REST API:** `curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/events/?severity=ERROR&from=2026-08-09T10:00:00' | python3 -m json.tool`

Ubuntu Log Correlation

 **PURPOSE:** Correlate Ubuntu client logs with VAST cluster events to find root cause.

```
# Ubuntu kernel NFS messages during incident window
dmesg -T | grep -iE 'nfs|error|fail|timeout' | tail -20

# Ubuntu system log
sudo grep -i nfs /var/log/syslog | tail -20

# Build merged timeline
dmesg -T | grep -iE 'nfs|error|fail' > /tmp/ubuntu_events.txt
echo '--- Ubuntu events ---'
cat /tmp/ubuntu_events.txt
echo '--- VAST events (from vms events list) ---'
echo 'Run: vms events list --from <incident-time> on management node'
```

✓ EXPECTED OUTPUT: Ubuntu NFS errors correlated with VAST cluster events at matching timestamps

Snapshot Commands — Backup and Recovery

🎯 PURPOSE: Use snapshots to recover from accidental deletion, corruption, or ransomware.

```
# List all snapshots
vms snapshots list

# Create manual snapshot before risky operation
vms snapshots create \
  --name pre_upgrade_snap \
  --path /ai_training

# Show snapshot details
vms snapshots show --name pre_upgrade_snap

# List snapshot policies
vms snapshotpolicies list

# Create automated daily snapshot policy
vms snapshotpolicies create \
  --name daily_ai_snap \
  --every DAY \
  --keep-count 7

# Attach snapshot policy to volume
vms volumes modify --name ai_training --snapshot-policy daily_ai_snap

# Show snapshots for a path (for recovery)
vms snapshots list

# Delete snapshot
vms snapshots delete --name pre_upgrade_snap
```

✓ EXPECTED OUTPUT: id | name | path | created_at | size | for each snapshot

🌐 REST API: `curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/snapshots/ | python3 -m json.tool`

Failure Pattern Analysis

🎯 PURPOSE: Identify recurring patterns to prevent future incidents.

```
# Check drive failure history
vms drives list
vms events list --object-type DRIVE --severity ERROR

# Check CNode event history
vms events list --object-type CNODE

# Check DBox events
vms events list --object-type DBOX

# Count events by type
vms events list --severity ERROR
```

```
# Create support bundle for pattern analysis
vms supportbundles create --reason 'Recurring drive failures pattern analysis'
vms supportbundles list
vms supportbundles download --id 1 --output /tmp/pattern_bundle.tar.gz
```

✓ EXPECTED OUTPUT: Drive events | CNode events | DBox events | Support bundle for VAST team

CAPSTONE SCENARIO 1 — AI Training Performance Degradation

🔴 **SCENARIO: Training job: 3 hours → 6 hours | GPU: 95% → 45% | Dataset loading slow | No code changes**

SCENARIO 1 DIAGNOSIS FLOW:

GPU utilization dropped from 95% to 45%

GPU is WAITING for data from storage

VAST cluster unhealthy?
vms alerts list CRITICAL
vms cnodes list
All ONLINE?

Fix CNode/DBox issue

Network degraded?
nfsstat retransmits > 0?
iperf3 bandwidth drop?

Fix network first

QoS policy throttling?
vms qosolicies list
Is wrong QoS attached to view?
Remove or increase QoS limit

Mount options wrong?

nconnect = 1?
rsize = 256KB?
noatime missing?

Remount with tuned options

Phase 1: Check VAST Cluster Health First

🎯 **PURPOSE: Always check cluster health before blaming network or client. Quick 2-minute check.**

```
echo '=== PHASE 1: VAST Cluster Health ==='

# Check cluster state
vms cluster show

# Check for critical alerts
vms alerts list --severity CRITICAL

# Check all CNodes online
vms cnodes list

# Check all DBoxes online
vms dboxes list

# Check current performance metrics
vms metrics get

echo 'If all ONLINE and no CRITICAL alerts -> cluster is healthy'
echo 'Move to Phase 2: check client mount options'
```

✓ EXPECTED OUTPUT: Cluster: ONLINE | No CRITICAL alerts | All CNodes ONLINE | All DBoxes ONLINE

Phase 2: Check Mount Options from Ubuntu

🎯 PURPOSE: Check if nconnect and rsize are correctly set. Missing nconnect is the #1 cause of AI training slowdowns.

```
echo '=== PHASE 2: Mount Options Check ==='

# Check current mount options
findmnt -t nfs,nfs4 -o TARGET,SOURCE,OPTIONS

# Count TCP connections — should be 16 for AI workloads
ss -tn dst :2049 | wc -l | xargs echo 'NFS TCP connections:'

# Check NFS retransmits
nfsstat -c 2>/dev/null | head -6

echo 'If connections = 1 -> nconnect not set -> this is the cause'
echo 'If retrans > 0 -> network issue -> check Phase 3'
```

✓ EXPECTED OUTPUT: OPTIONS shows nconnect=16 (good) OR nconnect missing/=1 (problem found)

Phase 3: Check Network

🎯 PURPOSE: If mount options are correct and retransmits > 0 — network is the problem.

```
echo '=== PHASE 3: Network Check ==='

# Latency to CNode
ping -c 10 <cnode-ip> 2>/dev/null | tail -2

# Bandwidth to CNode
iperf3 -c <cnode-ip> -t 10 -P 4 2>&1 | grep -E 'Gbits|Mbits' | tail -2

# NIC error check on Ubuntu
ethtool -S eth0 2>/dev/null | grep -E 'error|drop' | grep -v ' 0$'
```

✓ EXPECTED OUTPUT: ping < 1ms | iperf3 > 10 Gbits | No NIC errors = network healthy

Phase 4: Check QoS Policy on VAST

🎯 PURPOSE: A wrongly-applied QoS policy can cap throughput below what AI training needs.

```
echo '=== PHASE 4: QoS Policy Check ==='

# List QoS policies
vms qospolicies list

# Show which policy is attached to AI view
vms views show --name ai_training_view

# If wrong policy is attached — fix it
vms views modify --name ai_training_view --qos-policy ai_training_qos
```



```
# Or remove QoS entirely (uncapped)
vms views modify --name ai_training_view --qos-policy ''
```

✓ EXPECTED OUTPUT: QoS policy check | Correct policy attached | Or policy removed = uncapped

Phase 5: Apply Fix — Remount with Correct Options

🎯 PURPOSE: Fix the mount options. This resolves the most common cause of AI training slowdowns.

🔧 FIX: **nconnect=16 + rsize=1MB + noatime** = restores 3-hour training time

```
echo '=== PHASE 5: Apply Fix ==='

# Unmount
sudo umount -f -l /mnt/vast-ai

# Remount with correct options
# nconnect=16 = 16 TCP connections (was 1 = 16x slower)
# rsize=1M = 1MB read block (was 256KB = 4x more trips)
# noatime = skip access time metadata write per read
sudo mount -t nfs \
  -o vers=3,nconnect=16,rsize=1048576,wsiz=1048576,noatime,hard,proto=tcp \
  <cnode-ip>:/ai_training /mnt/vast-ai

# Verify options
findmnt /mnt/vast-ai -o TARGET,OPTIONS

# Verify 16 connections
ss -tn dst :2049 | wc -l | xargs echo 'TCP connections after fix:'

echo '=== PHASE 5: Verify Fix ==='

# Measure throughput after fix
echo 'BEFORE (single connection):'
fio --name=before --rw=read --bs=256k --size=512M \
  --directory=/mnt/single --numjobs=1 --group_reporting \
  2>&1 | grep bw=

echo 'AFTER (nconnect=16, 1MB blocks):'
fio --name=after --rw=read --bs=1M --size=512M \
  --directory=/mnt/vast-ai --numjobs=4 --group_reporting \
  2>&1 | grep bw=
```

✓ EXPECTED OUTPUT: TCP connections: 16 | BEFORE: low MB/s | AFTER: much higher MB/s | Training time restored

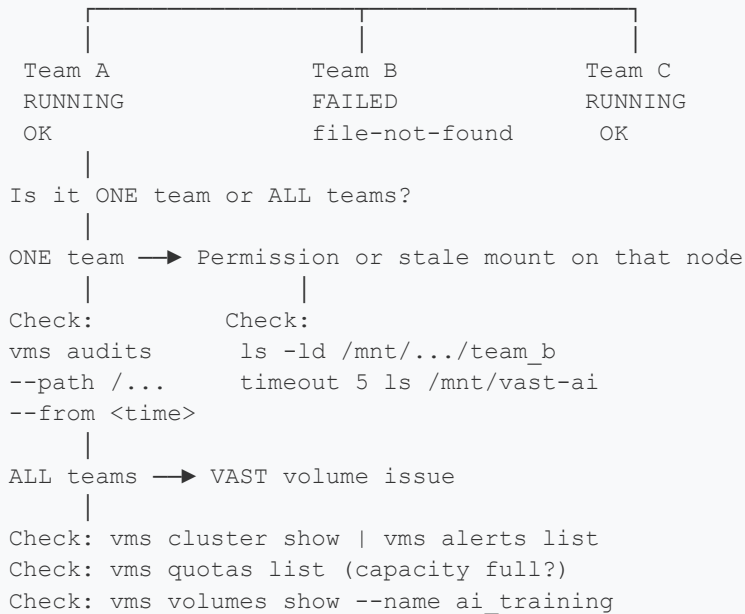
💡 KEY LESSON: GPU utilization = storage throughput delivered. If GPU drops, check storage FIRST. **nconnect=16** and **rsize=1M** are the two commands that fix 90% of AI training performance issues on VAST NFS.

CAPSTONE SCENARIO 2 — AI Dataset Access Failure

🔴 **SCENARIO: Intermittent file-not-found | Slow dataset browsing | Some teams fail, others succeed | Storage growing**

SCENARIO 2 DIAGNOSIS FLOW:

Shared Dataset: /ai_training/datasets/



Phase 1: Scope the Problem on VAST

🎯 **PURPOSE: Determine if this is a VAST cluster issue or a specific-team issue.**

```
echo '=== PHASE 1: Scope Assessment ==='

# Check cluster health
vms cluster show
vms alerts list --severity CRITICAL

# Check volume exists and is accessible
vms volumes show --name ai_training

# Check quota - is volume full?
vms quotas list

# Check audit log - who got permission denied?
vms audits list --path /ai_training --from 2026-08-09T00:00:00

echo 'If cluster healthy + quota OK + one team in audit denials'
echo '-> Permission issue for that team only -> Phase 2'
echo 'If quota 100% -> capacity issue -> Phase 4'
```

✅ **EXPECTED OUTPUT: Cluster ONLINE | Volume exists | Quota 20% used | Audit shows team_b denied**

Phase 2: Permission Failure Diagnosis on VAST

🎯 **PURPOSE:** Find and fix permission denied for one team while others work.

```
echo '=== PHASE 2: Permission Check ==='

# Check user exists and is in correct group
vms users show --name team_b_user
vms usergroups show --name ai_team

# Check audit log for team_b specifically
vms audits list --path /ai_training

# From Ubuntu: check directory permissions
ls -ld /mnt/vast-ai/datasets/team_a
ls -ld /mnt/vast-ai/datasets/team_b
ls -ld /mnt/vast-ai/datasets/team_c

# From Ubuntu: test access as each team
ls /mnt/vast-ai/datasets/team_a/ && echo 'team_a: OK'
ls /mnt/vast-ai/datasets/team_b/ 2>&1 || echo 'team_b: FAILED'
ls /mnt/vast-ai/datasets/team_c/ && echo 'team_c: OK'

# Fix: restore permissions on team_b directory
sudo chown $USER:$USER /mnt/vast-ai/datasets/team_b
sudo chmod 755 /mnt/vast-ai/datasets/team_b
ls /mnt/vast-ai/datasets/team_b/ && echo 'team_b: FIXED'
```

✅ **EXPECTED OUTPUT:** team_a: OK | team_b: FAILED then FIXED | team_c: OK

Phase 3: Stale Mount Diagnosis

🎯 **PURPOSE:** One node sees file-not-found while others can access. Classic stale NFS mount.

```
echo '=== PHASE 3: Stale Mount Check ==='

# Test for stale mount
timeout 5 ls /mnt/vast-ai/datasets/ 2>&1 || echo 'STALE: file not found'

# Recovery: force unmount
sudo umount -f -l /mnt/vast-ai

# Restart NFS server (if local lab)
sudo systemctl start nfs-kernel-server 2>/dev/null

# Remount
sudo mount -t nfs \
  -o vers=3,nconnect=16,rsz=1048576,wsz=1048576,noatime,hard,proto=tcp \
  <cnode-ip>:/ai_training /mnt/vast-ai

# Verify recovery
ls /mnt/vast-ai/datasets/ && echo 'RECOVERED: namespace accessible'

# VAST side: check CNode was available
vms cnodes list
vms events list --severity ERROR --from 2026-08-09T00:00:00
```

✓ EXPECTED OUTPUT: Stale detected | Unmounted | Remounted | RECOVERED | CNode events
show restart

Phase 4: Capacity Issue Diagnosis

🎯 PURPOSE: Storage utilization growing + jobs failing = quota or volume capacity issue.

```
echo '=== PHASE 4: Capacity Check ==='

# Check VAST volume capacity
vms volumes show --name ai_training

# Check quota usage
vms quotas list
vms quotas show --name team_a_quota

# From Ubuntu: check capacity
df -h /mnt/vast-ai
USED=$(df /mnt/vast-ai 2>/dev/null | awk 'NR==2{print $5}' | tr -d '%')
echo "Storage usage: ${USED}%"

# Find what is consuming space
du -sh /mnt/vast-ai/* 2>/dev/null | sort -rh | head -10

# Find largest files (old checkpoints)
find /mnt/vast-ai -type f -size +1G \
    -exec ls -lh {} \; 2>/dev/null | sort -k5 -rh | head -10

# Fix: increase quota if needed
vms quotas modify --name team_a_quota --hard-limit 20TB

# Or clean old checkpoints from Ubuntu
# Identify then remove - always verify before deleting
find /mnt/vast-ai/datasets -name '*.pt' -mtime +30 | head -5
```

✓ EXPECTED OUTPUT: Quota usage shown | Large files identified | Quota increased OR old files cleaned

Phase 5: Slow Dataset Browsing Diagnosis

🎯 PURPOSE: ls is slow on dataset directory = too many files or high NFS metadata load.

```
echo '=== PHASE 5: Slow Browsing Check ==='

# Count files in dataset directory
ls /mnt/vast-ai/datasets/team_a/ | wc -l | xargs echo 'Files in dir:'

# Check NFS metadata operations
echo 'NFS stats before ls:'
nfsstat -c 2>/dev/null | head -4
ls /mnt/vast-ai/datasets/ > /dev/null
echo 'NFS stats after ls (readdir should have jumped):'
nfsstat -c 2>/dev/null | head -4

# VAST side: check audit for metadata operations
vms audits list --path /ai_training --from 2026-08-09T00:00:00
```

✔ EXPECTED OUTPUT: File count shown | readdir counter jumped | Audit shows high getattr/readdir operations

Full Scenario 2 Resolution Summary

Symptom	VAST Command	Ubuntu Command	Fix
team_b denied	vms audits list --path /ai_training	ls -ld /mnt/.../team_b	sudo chown \$USER + chmod 755
Stale mount	vms events list --severity ERROR	timeout 5 ls /mnt/vast-ai	umount -f -l + remount
Capacity full	vms quotas list	df -h /mnt/vast-ai	vms quotas modify --hard-limit 20TB
Old checkpoints	vms volumes show --name ai_training	du -sh /mnt/vast-ai/* sort -rh	find + rm old .pt files
Slow browsing	vms audits list --path /path	nfsstat -c (readdir count)	Split dataset into subdirs

💡 KEY LESSON: Intermittent = isolate systematically. One team failing = permission or stale mount. All teams failing = quota or VAST cluster issue. Always check vms audits list for the actual denied operation before guessing.

COMPLETE VAST COMMAND QUICK REFERENCE

Module	VAST Command	Purpose
1	vms cluster show	Full cluster overview
1	vms cnodes list	List compute nodes + IPs
1	vms dboxes list	List flash enclosures
1	vms drives list	List individual SSDs
1	vms volumes list	List data volumes
1	vms volumes create --name X --path /X	Create volume
1	vms volumes show --name X	Volume details + usage
1	vms volumes modify --name X --snapshot-policy Y	Attach snapshot policy
1	vms volumes delete --name X	Delete volume
1	vms views list	List all protocol exports
1	vms views create --name X --path /X --protocols NFS	Create NFS export
1	vms views create --name X --path /X --protocols SMB	Create SMB share
1	vms views create --name X --path /X --protocols NFS,SMB,S3	Multi-protocol view
1	vms views show --name X	View details + NFS path
1	vms views modify --name X --qos-policy Y	Attach QoS to view
1	vms views delete --name X	Delete view
2	vms alerts list	All active alerts
2	vms alerts list --severity CRITICAL	Critical alerts only
2	vms alerts show --id N	Alert details
2	vms events list	All events
2	vms events list --severity ERROR	Error events
2	vms events list --from YYYY-MM-DDTHH:MM:SS	Events since time
2	vms metrics get	Current perf metrics
2	vms metrics get --format csv	Metrics as CSV
3	vms qosolicies list	List QoS policies
3	vms qosolicies create --name X --max-throughput-mbps N --max-iops N	Create QoS
3	vms qosolicies show --name X	QoS details
3	vms qosolicies delete --name X	Delete QoS
4	vms nics list	List network interfaces
4	vms nics show --id N	NIC details
5 NFS	vms views create --protocols NFSv3	NFSv3 only export
5 NFS	vms views create --protocols NFSv4	NFSv4 only export
5 SMB	vms smbshares list	List SMB shares
5 SMB	vms users list	List users
5 SMB	vms users create --name X --password Y	Create user
5 SMB	vms usergroups list	List groups
5 SMB	vms usergroups create --name X	Create group
5 SMB	vms usergroups modify --name X --add-users Y	Add user to group
5 SMB	vms audits list	Access audit log
5 SMB	vms audits list --protocol SMB	SMB audit only
5 SMB	vms audits list --protocol NFS	NFS audit only

6	vms protectionpolicies list	List data reduction policies
6	vms protectionpolicies create --name X --compression false --dedup false	No reduction
6	vms protectionpolicies create --name X --compression true --dedup true	Max reduction
6	vms volumes modify --name X --protection-policy Y	Attach protection policy
6	vms quotas list	List quotas
6	vms quotas create --name X --path /Y --hard-limit NTB	Create quota
6	vms quotas modify --name X --hard-limit NTB	Increase quota
6	vms quotas show --name X	Quota usage %
7	vms alerts list --severity CRITICAL	Critical alert triage
7	vms events list --severity ERROR	Error event triage
7	vms supportbundles create --reason 'text'	Create support bundle
7	vms supportbundles list	List bundles
7	vms supportbundles download --id N --output /path/file.tar.gz	Download bundle
8	vms metrics get	Perf metrics for diagnosis
8	vms qosolicies show --name X	Check QoS limits
8	vms quotas show --name X	Check quota headroom
9	vms snapshots list	List snapshots
9	vms snapshots create --name X --path /Y	Create manual snapshot
9	vms snapshotpolicies list	List snapshot policies
9	vms snapshotpolicies create --name X --every DAY --keep-count 7	Daily auto-snapshots
9	vms snapshots delete --name X	Delete snapshot
Cap 1	vms cluster show + vms alerts list	First check for perf issue
Cap 1	vms qosolicies list + vms views show	Check QoS throttling
Cap 2	vms audits list --path /path	Find denied operations
Cap 2	vms quotas list	Capacity check
Cap 2	vms volumes show --name X	Volume usage

REST API Endpoint	Method	Use For
/api/clusters/	GET	Module 1,2 — cluster health
/api/cnodes/	GET	Module 1,4 — CNode IPs and status
/api/dboxes/	GET	Module 1 — DBox status
/api/drives/	GET	Module 1,7 — drive health
/api/folders/	GET/POST	Module 1 — volumes
/api/views/	GET/POST	Module 1,5 — NFS/SMB/S3 views
/api/qosolicies/	GET/POST	Module 3,6 — QoS
/api/protectionpolicies/	GET/POST	Module 6 — data reduction
/api/quotas/	GET/POST	Module 6,7,8 — capacity
/api/alerts/?resolved=false	GET	Module 2,7 — active alerts
/api/events/?severity=ERROR	GET	Module 2,8,9 — error events
/api/audits/?protocol=NFS	GET	Module 5,7 — access audit
/api/users/	GET/POST	Module 5 SMB — users
/api/usergroups/	GET/POST	Module 5 SMB — groups
/api/snapshots/	GET/POST	Module 9 — snapshots
/api/supportbundles/	GET/POST	Module 7,9 — support bundles
/api/monitors/ad hoc query/	POST	Module 2,3,8 — metrics
/api/nics/	GET	Module 4 — network interfaces